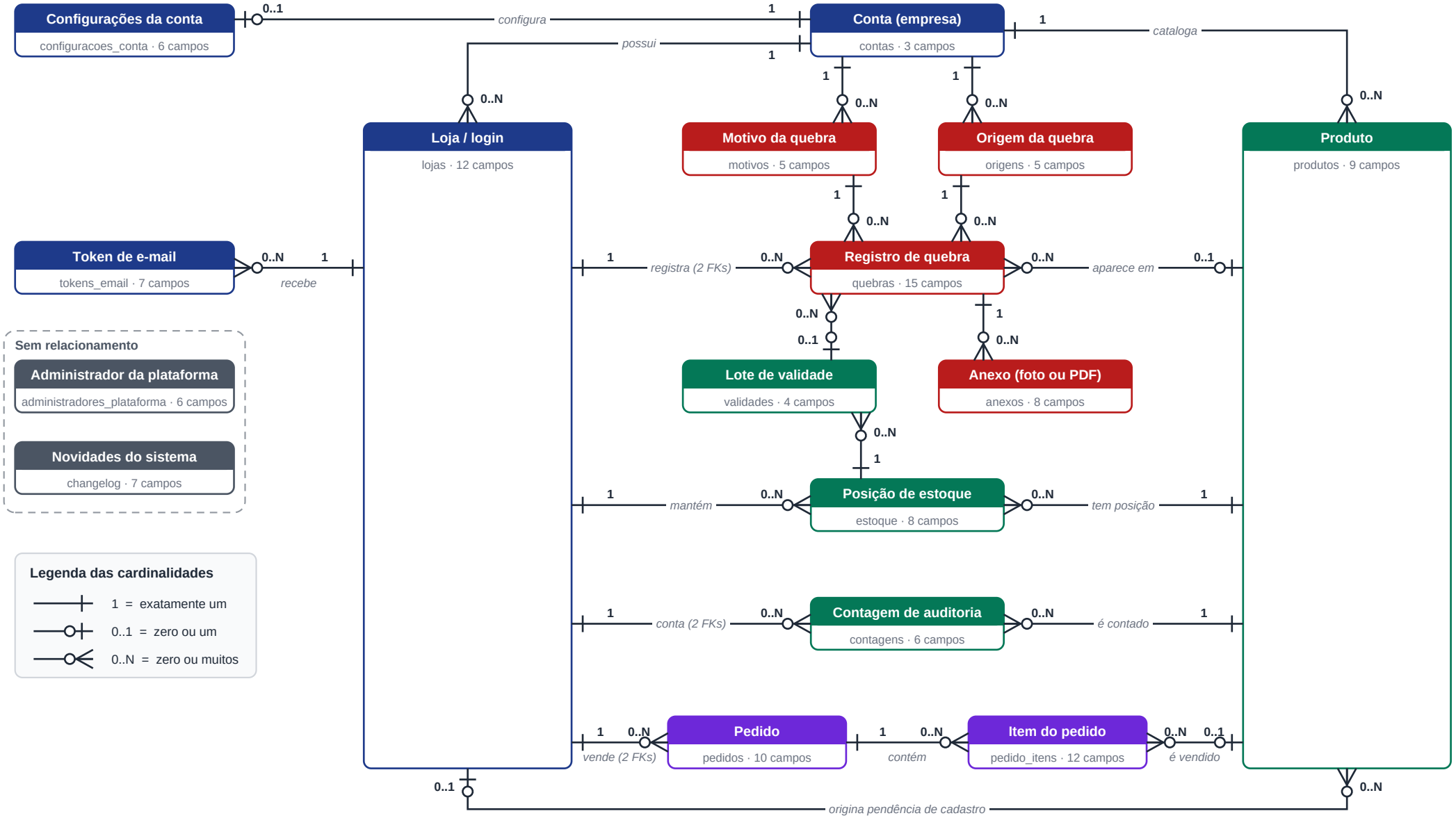
Gestão de Validades | Modelagem de dados

Visão geral: 16 tabelas em 5 módulos, com as chaves estrangeiras declaradas no banco. Os atributos e os vínculos inferidos estão nas páginas de cada módulo (2 a 5).

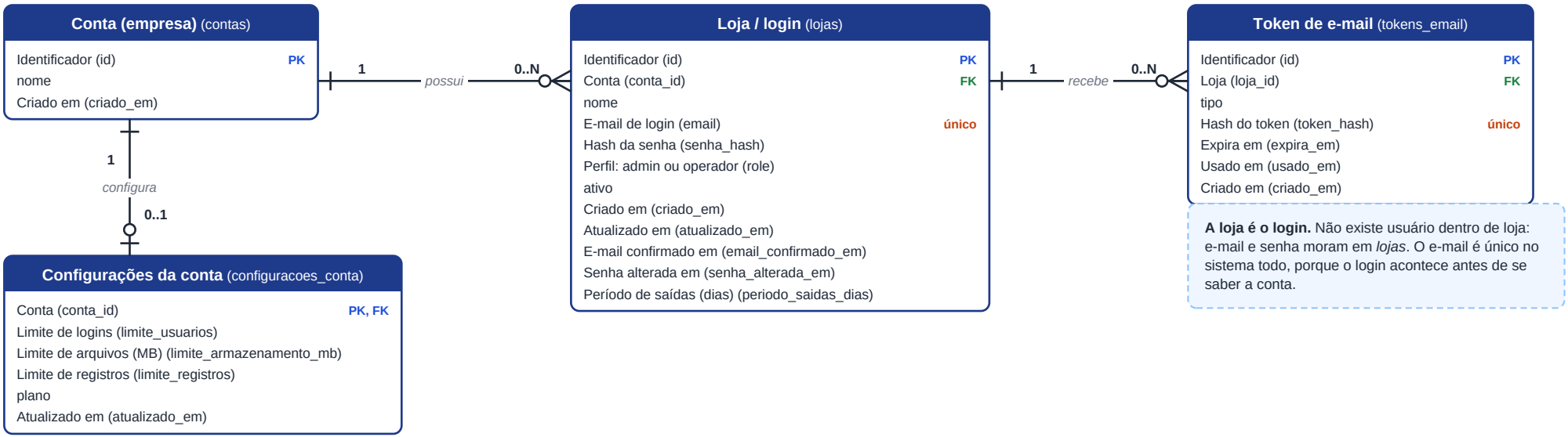
Conta e acesso · Catálogo, estoque e auditoria · Quebras · Pedidos · Plataforma (sem vínculo)



Regra central: tudo pertence a uma conta (empresa) e cada login é uma loja. O operador só enxerga a própria loja; o administrador, todas as lojas da conta. Quem garante é a RLS do banco.

Módulo 1 | Conta e acesso

A empresa (conta), os logins (cada login é uma loja), o plano e os tokens de e-mail. Abaixo, as duas tabelas da plataforma, que não se ligam a nenhuma outra.



A loja é o login. Não existe usuário dentro de loja: e-mail e senha moram em *lojas*. O e-mail é único no sistema todo, porque o login acontece antes de se saber a conta.

Outros módulos dependem daqui. A conta é referenciada por produtos, motivos e origens; a loja, por produtos (origem), estoque, contagens, quebras e pedidos. Esses vínculos estão desenhados nas páginas 3 a 5.

Plataforma: sem relacionamento com as outras tabelas

Administrador da plataforma (administradores_plataforma)

Identificador (id)	PK
nome	
E-mail (email)	único
Hash da senha (senha_hash)	
ativo	
Criado em (criado_em)	

Novidades do sistema (changelog)

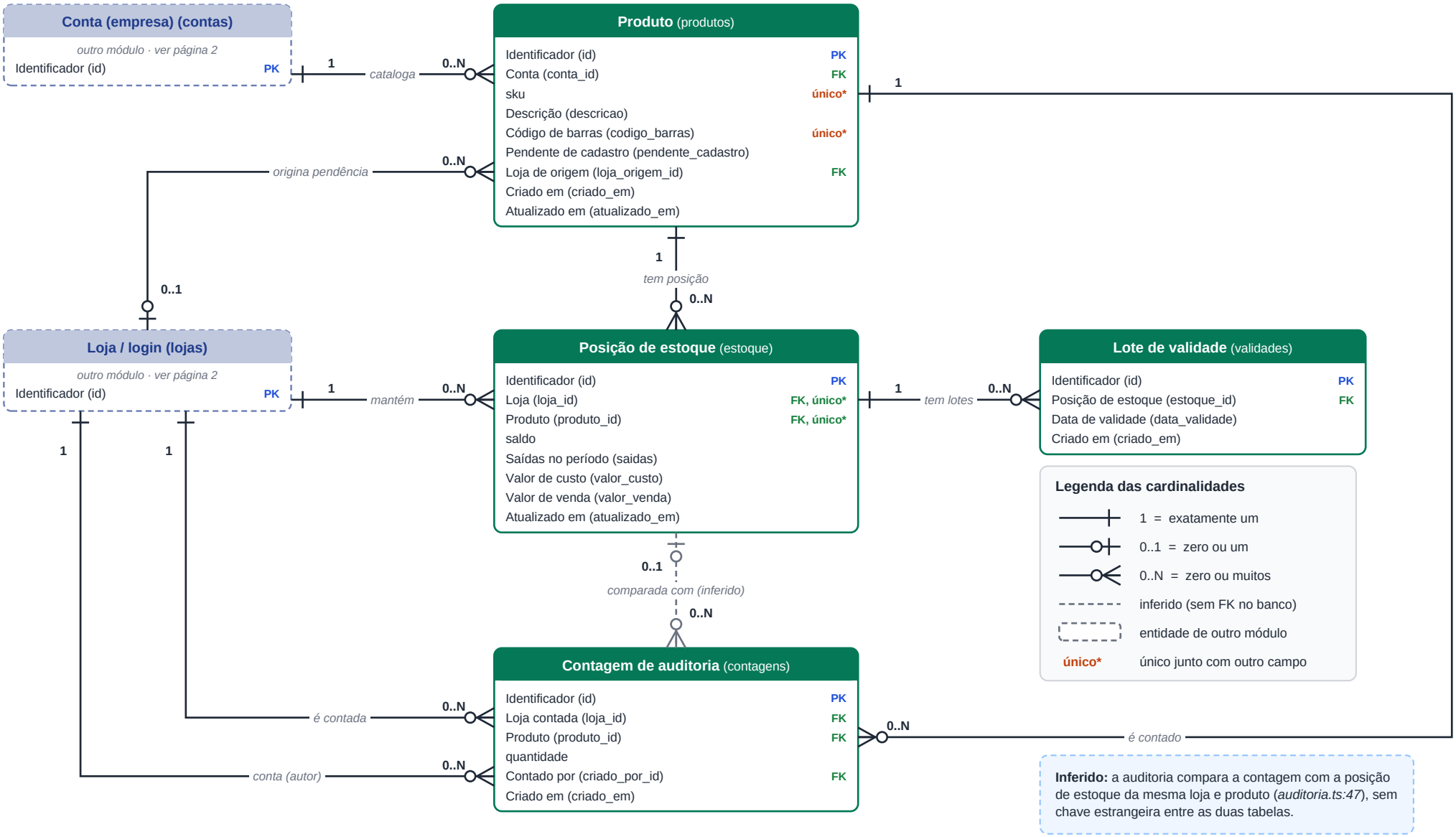
Identificador (id)	PK
Versão (versao)	
Título (titulo)	
Descrição (descricao)	
Só para admin (restrito_admin)	
Texto genérico (texto_generico)	
Publicado em (publicado_em)	

Legenda das cardinalidades

- 1 = exatamente um
- 0..1 = zero ou um
- 0..N = zero ou muitos
- inferido (sem FK no banco)
- entidade de outro módulo
- único* — único junto com outro campo

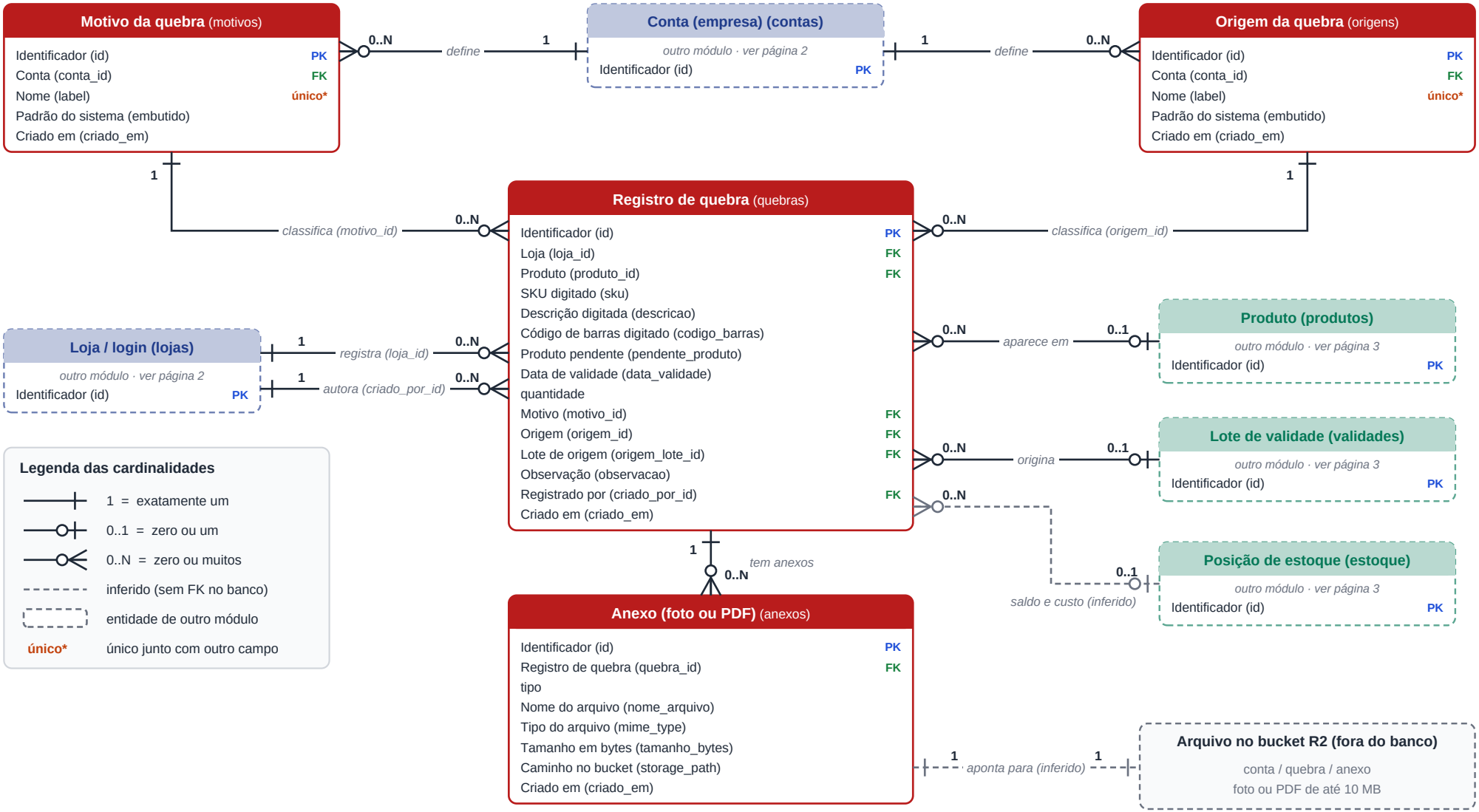
Módulo 2 | Catálogo, estoque e auditoria

O catálogo é da conta inteira; a posição de estoque e os lotes são de cada loja. A contagem de auditoria soma parcelas por loja e produto.



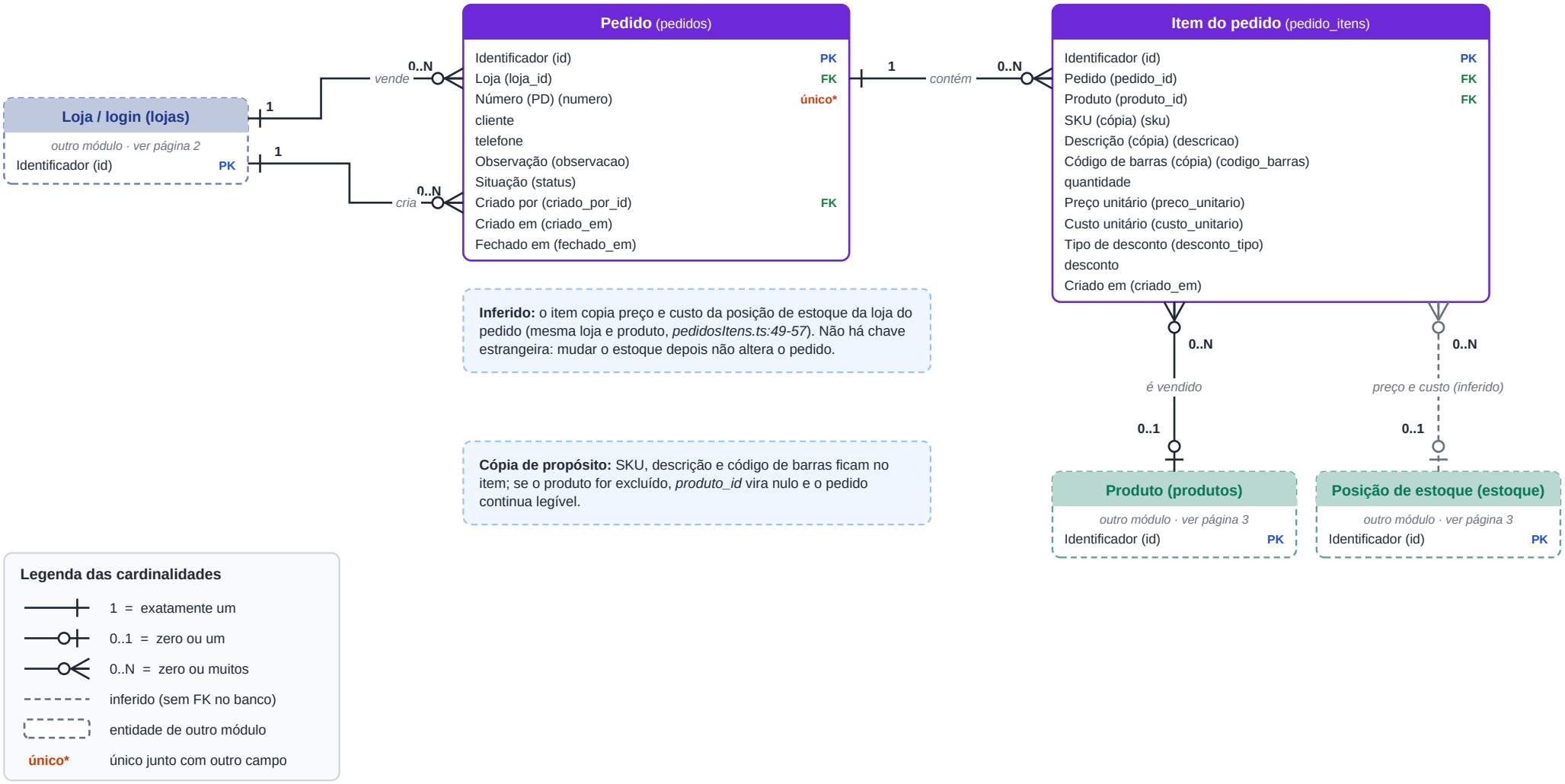
Módulo 3 | Quebras

O registro de perda de uma loja, com motivo e origem obrigatórios, e os anexos (fotos e PDFs). O arquivo em si fica fora do banco, no bucket R2.



Módulo 4 | Pedidos

O pedido de um cliente (PD), numerado por loja, e os itens. Preço e custo são copiados do estoque da loja quando o item entra, e não mudam depois.



Relacionamentos e regras de integridade

Leitura de cada relacionamento, como está no código (backend/migrations M0 a M9 e rotas em backend/src/rotas). Linhas em cinza são vínculos inferidos, sem chave estrangeira.

Relacionamento	Chave	Cardinalidade	Leitura
Conta → Loja / login	lojas.conta_id	**1 : 0..N**	Uma conta possui nenhuma ou várias lojas (logins). Cada loja pertence a exatamente uma conta.
Conta → Configurações da conta	configuracoes_conta.conta_id (PK)	**1 : 0..1**	Uma conta tem nenhuma ou uma linha de configurações. Cada configuração pertence a exatamente uma conta.
Conta → Produto	produtos.conta_id	**1 : 0..N**	Uma conta cataloga nenhum ou vários produtos. Cada produto pertence a exatamente uma conta.
Conta → Motivo da quebra	motivos.conta_id	**1 : 0..N**	Uma conta define nenhum ou vários motivos. Cada motivo pertence a exatamente uma conta.
Conta → Origem da quebra	origens.conta_id	**1 : 0..N**	Uma conta define nenhuma ou várias origens. Cada origem pertence a exatamente uma conta.
Loja / login → Token de e-mail	tokens_email.loja_id	**1 : 0..N**	Uma loja recebe nenhum ou vários tokens de e-mail. Cada token é de exatamente uma loja.
Loja / login → Produto (loja de origem)	produtos.loja_origem_id	**0..1 : 0..N**	Uma loja origina nenhum ou vários produtos pendentes. Um produto tem nenhuma ou uma loja de origem.
Loja / login → Posição de estoque	estoque.loja_id	**1 : 0..N**	Uma loja mantém nenhuma ou várias posições de estoque. Cada posição é de exatamente uma loja.
Produto → Posição de estoque	estoque.produto_id	**1 : 0..N**	Um produto tem nenhuma ou várias posições (uma por loja). Cada posição é de exatamente um produto.
Posição de estoque → Lote de validade	validades.estoque_id	**1 : 0..N**	Uma posição de estoque tem nenhum ou vários lotes. Cada lote pertence a exatamente uma posição.
Loja / login → Contagem (loja contada)	contagens.loja_id	**1 : 0..N**	Uma loja é contada em nenhuma ou várias parcelas. Cada parcela é de exatamente uma loja.
Loja / login → Contagem (autor)	contagens.criado_por_id	**1 : 0..N**	Um login registra nenhuma ou várias parcelas. Cada parcela tem exatamente um autor.
Produto → Contagem de auditoria	contagens.produto_id	**1 : 0..N**	Um produto é contado em nenhuma ou várias parcelas. Cada parcela é de exatamente um produto.
Loja / login → Registro de quebra (dona)	quebras.loja_id	**1 : 0..N**	Uma loja registra nenhuma ou várias quebras. Cada quebra é de exatamente uma loja.
Loja / login → Registro de quebra (autor)	quebras.criado_por_id	**1 : 0..N**	Um login é autor de nenhuma ou várias quebras. Cada quebra tem exatamente um autor.
Produto → Registro de quebra	quebras.produto_id	**0..1 : 0..N**	Um produto aparece em nenhuma ou várias quebras. Uma quebra tem nenhum ou um produto do cadastro.
Motivo da quebra → Registro de quebra	quebras.motivo_id	**1 : 0..N**	Um motivo classifica nenhuma ou várias quebras. Cada quebra tem exatamente um motivo.
Origem da quebra → Registro de quebra	quebras.origem_id	**1 : 0..N**	Uma origem classifica nenhuma ou várias quebras. Cada quebra tem exatamente uma origem.
Lote de validade → Registro de quebra	quebras.origem_lote_id	**0..1 : 0..N**	Um lote origina nenhuma ou várias quebras. Uma quebra tem nenhum ou um lote de origem.
Registro de quebra → Anexo	anexos.quebra_id	**1 : 0..N**	Uma quebra tem nenhum ou vários anexos. Cada anexo pertence a exatamente uma quebra.
Loja / login → Pedido (loja que vende)	pedidos.loja_id	**1 : 0..N**	Uma loja vende nenhum ou vários pedidos. Cada pedido é de exatamente uma loja.
Loja / login → Pedido (autor)	pedidos.criado_por_id	**1 : 0..N**	Um login cria nenhum ou vários pedidos. Cada pedido tem exatamente um autor.
Pedido → Item do pedido	pedido_itens.pedido_id	**1 : 0..N**	Um pedido contém nenhum ou vários itens. Cada item pertence a exatamente um pedido.
Produto → Item do pedido	pedido_itens.produto_id	**0..1 : 0..N**	Um produto é vendido em nenhum ou vários itens. Um item tem nenhum ou um produto (nulo se o produto for excluído).
Posição de estoque → Registro de quebra *(inferido)*	loja_id + produto_id (consulta)	**0..1 : 0..N**	Uma posição de estoque fornece saldo e custo a nenhuma ou várias quebras da mesma loja e produto. Uma quebra corresponde a nenhuma ou uma posição.
Posição de estoque → Contagem de auditoria *(inferido)*	loja_id + produto_id (consulta)	**0..1 : 0..N**	Uma posição de estoque é comparada com nenhuma ou várias parcelas da mesma loja e produto.
Posição de estoque → Item do pedido *(inferido)*	pedidos.loja_id + produto_id (cópia)	**0..1 : 0..N**	Uma posição de estoque fornece preço e custo a nenhum ou vários itens no momento em que entram.
Anexo → Arquivo no bucket R2 *(inferido)*	anexos.storage_path	**1 : 1**	Cada anexo aponta para exatamente um arquivo no bucket, e cada arquivo é de exatamente um anexo.

Regras de integridade e onde cada uma é garantida

Banco = constraint, gatilho, função ou RLS. Back-end = código das rotas. Só front-end = a tela avisa ou impede, mas a API aceita.

Unicidade	Onde é garantida	Origem no código
E-mail de login único no sistema todo (não por conta).	Banco	M0 lojas.email unique
SKU único por conta; código de barras único por conta quando preenchido.	Banco	M0 unique (conta_id, sku); índice parcial (conta_id, codigo_barras)
Uma posição de estoque por loja e produto.	Banco	M0 unique (loja_id, produto_id)
Nome de motivo e de origem único por conta.	Banco	M0 unique (conta_id, label)
Número do pedido único e sequencial por loja.	Banco	M8 unique (loja_id, numero) + gatilho pedidos_numerar
E-mail do administrador da plataforma e hash de token de e-mail únicos.	Banco	M0, M3 unique
Um lote por posição de estoque e data de validade.	Back-end	validades.ts:110-119 (trava, sem índice único)
Um anexo por tipo em cada quebra (o novo substitui o antigo).	Back-end	anexos.ts:113

Obrigatoriedade e valores válidos	Onde é garantida	Origem no código
Motivo e origem obrigatórios em toda quebra.	Banco Back-end	M0 not null + quebras.ts:123
Saldo, saídas, custo, venda, preço e desconto nunca negativos.	Banco Back-end	M0, M8 check (>= 0)
Quantidade: quebra ≥ 0 no banco e na API; a tela exige ≥ 1.	Banco Back-end Front-end	M0 check; quebras.ts:100; QuantityField.tsx:46
Item do pedido: quantidade de 1 a 1.000.000; desconto não passa do valor da linha.	Banco	M8 check (desconto_tipo, desconto)
Telefone do pedido só com dígitos (10 ou 11) ou vazio.	Banco Back-end	M8 check; pedidos.ts:78-82
Pedido fechado ⇔ data de fechamento preenchida.	Banco	M8 check (status, fechado_em)
Período de saídas do login entre 1 e 3650 dias.	Banco Back-end	M5 check; conta.ts
Senha nova com 8+ caracteres, letras e números, até 72 bytes.	Back-end Front-end	senha.ts:20; validation.ts:29
Anexo só JPG, PNG, WebP, HEIC, HEIF ou PDF, até 10 MB.	Back-end	anexos.ts:28-31,80
Item só entra com preço de venda > 0 no estoque da loja; o preço não muda ao editar.	Back-end	pedidosItens.ts:49-60,88
Limites de registros e de MB por conta, conferidos a cada inclusão.	Banco	M6 gatilhos; M7; M8
Limite de logins ativos por conta (inclusive ao reativar).	Back-end	lojas.ts:41-56

Exclusão	Onde é garantida	Origem no código
Apagar a conta apaga em cascata lojas, produtos, motivos, origens e configurações.	Banco	M0 on delete cascade; M9 excluir_conta
Apagar a loja apaga estoque, tokens, contagens e pedidos; apagar a posição apaga os lotes; apagar a quebra apaga os anexos; apagar o pedido apaga os itens.	Banco	M0, M3, M7, M8 on delete cascade
Bloqueado: produto com estoque; motivo ou origem em uso; loja com quebras (como dona ou autora) ou com contagens e pedidos de sua autoria.	Banco	M0, M7, M8 on delete restrict
Vira nulo: produto da quebra e do item, lote de origem da quebra, loja de origem do produto.	Banco	M0, M8 on delete set null
Pedido fechado não se altera nem se exclui; é preciso reabrir antes.	Back-end	lib/pedidos.ts:85-93; pedidos.ts:242-259
Excluir todo o estoque exige digitar uma palavra de confirmação.	Só front-end	DeleteStockDialog.tsx:17,134
Excluir a empresa exige o nome dela (e a senha, no caso do cliente); apaga também os arquivos.	Back-end Front-end	conta.ts; plataforma.ts; exclusaoDeConta.ts

Acesso	Onde é garantida	Origem no código
Uma conta nunca vê dados de outra; o operador só vê a própria loja.	Banco	M1 RLS forçada em todas as tabelas
Só o administrador cria, altera e exclui produtos; o operador só cria produto pendente.	Banco Back-end	M1:155-163; M2; produtos.ts
Perfil, e-mail e conta do login não mudam depois de criados.	Banco	M4 privilégio por coluna
O administrador não desativa o próprio login.	Back-end	lojas.ts:126-129

Pontos de atenção

Descrição do estado atual, sem propostas de correção.

Tabelas e campos sem relacionamento ou sem uso

- **administradores_plataforma** e **changelog** não se ligam a nenhuma outra tabela. O *changelog* não é lido nem escrito por nenhuma rota ou tela.
- **quebras.origem_lote_id** tem chave estrangeira, mas nenhuma rota ou tela grava ou lê o campo.
- Todo campo terminado em *_id* tem chave estrangeira declarada; os vínculos tracejados do diagrama são ligações feitas só por consulta, listadas na seção "Inferidos".

Regras que existem só no front-end (ou só no back-end)

- **Só na tela:** quantidade de quebra a partir de 1 (a API aceita 0); aviso de quantidade acima do saldo, na quebra e no pedido (nada impede); palavra de confirmação para excluir todo o estoque; a situação do lote (venceu, vence antes de vender, vende antes, sem estimativa) é calculada só na tela e não é gravada; a redução da foto antes do envio.
- **Só no back-end, sem garantia no banco:** lote único por data; pedido fechado imutável; pedido sem itens não fecha; limite de logins; um anexo por tipo; tipos e tamanho de arquivo; editar produto encerra a pendência; o administrador não desativa o próprio login.

Campos sem restrição no banco

- Textos sem limite de tamanho nem formato: *contas.nome*, *lojas.nome*, *lojas.email* (sem verificação de formato nem de minúsculas), *produtos.sku*, *produtos.descricao*, *produtos.codigo_barras*, *motivos.label*, *origens.label*, *quebras.sku*, *quebras.descricao*, *quebras.observacao*, *anexos.nome_arquivo*, *anexos.mime_type* e *anexos.storage_path*. Os limites existem só nas rotas.
- *configuracoes_conta.limite_usuarios* e *limite_armazenamento_mb* aceitam zero ou negativo no banco (só *limite_registros* tem verificação).
- *validades.data_validade* é opcional e pode se repetir na mesma posição de estoque.
- *quebras.sku*, *descricao* e *codigo_barras* são cópias digitadas, sem vínculo com o cadastro.

Dados sensíveis

- **Imagens e documentos:** fotos de produto e de etiqueta e PDFs dos anexos ficam no bucket R2 da Cloudflare, fora do banco; o banco guarda nome, tipo, tamanho e caminho. Antes do envio, o rascunho e as fotos ficam no aparelho (localStorage e IndexedDB). Fotos que não passam pela redução podem levar a localização (EXIF) — inferência a partir de *image.ts*.
- **Dados pessoais de clientes finais:** nome, telefone e observação livre em *pedidos*.
- **Dados de quem usa o sistema:** nome e e-mail em *lojas* e *administradores_plataforma*; senha guardada só como hash (bcrypt); tokens de e-mail só como hash (SHA-256).
- **Dados comerciais:** custo e preço de venda em *estoque* e *pedido_itens*.
- **Saúde e crianças:** o modelo não tem nenhum campo de saúde nem de dados de crianças.

Vínculos marcados como inferidos

- **Posição de estoque → Registro de quebra:** a lista de quebras busca saldo e custo pela mesma loja e produto (*quebras.ts:45-49*); não há chave estrangeira entre as duas tabelas.
- **Posição de estoque → Contagem de auditoria:** a auditoria junta as duas por loja e produto (*auditoria.ts:47*).
- **Posição de estoque → Item do pedido:** preço e custo são copiados do estoque da loja do pedido no momento em que o item entra (*pedidosItens.ts:49-57*).
- **Anexo → Arquivo no bucket R2:** o vínculo é o texto *storage_path* (conta/quebra/anexo, *anexos.ts:21-23*); o banco não sabe se o arquivo existe.